

毕业论文（设计）任务书

学生姓名	曹志隆	学号	32301288	专业 班级	软件工程 2304	电话	13868360863
毕业论文（设计）中文题目		面向超级个体的 AIGC 协作开发工作流平台设计与实现					
毕业论文（设计）外文题目		Design and Implementation of an AIGC-Driven Collaborative Development Workflow Platform for Solo Developers					
指导教师 1	侯宏仑	职称	副教授	所在单位	计算机与计算科学学院	电话	13071858629
指导教师 2		职称		所在单位		电话	
选题类型	√ 工程设计 ☐ 理论研究 ☐ 实验研究 ☐ 产品开发 ☐ 设计创作 ☐ 专题研究 ☐ 应用研究 ☐ 其他						
选题来源	☐ 国家重点研发计划项目 ☐ 国家社科规划、基金项目 ☐ 国家自然科学基金项目 ☐ 中央、国家各部门项目 ☐ 教育部人文、社会科学研究项目 ☐ 省（自治区、直辖市）项目 ☐ 国际合作研究项目 ☐ 港、澳、台合作研究项目 ☐ 企、事业单位委托项目 ☐ 外资项目 ☐ 国防项目 √ 学校自选项目 ☐ 非立项 ☐ 其他						
是否在实验、实习、工程实践和社会调查等社会实践中完成						√ 是 ☐ 否	
毕业论文（设计）时间		【待填：学院规定起止日期】					
阶段工作内容		工作进度安排周次					
查阅文献		第 5 周（第七学期）至第 7 周（第七学期）					
文献综述和外文翻译		第 8 周（第七学期）至第 12 周（第七学期）					
开题报告		第 13 周（第七学期）至第 17 周（第七学期）					
研究、设计、开发		第 1 周（第八学期）至第 7 周（第八学期）					
撰写毕业论文（说明）		第 8 周（第八学期）至第 12 周（第八学期）					
答辩与修改		第 12 周（第八学期）至第 15 周（第八学期）					
“阶段工作内容”包括：查阅文献、调研、撰写综述、翻译外文资料、开题、实验或设计、绘图、撰写毕业论文或设计说明书、定稿及答辩等；							
论文（设计）的主要内容及要求：（以设计作品为主要成果形式的毕业设计要求以各分院实施方案为准）							
1、开题报告和文献阅读							
(1) 文献阅读：查阅文献 15 篇（含）以上，其中外文文献 2 篇（含）以上，近三年公开发表的文献 5 篇（含）以上，书籍不超过 3 本，期刊〔J〕和论文集〔C〕10 篇（含）以上，包括导师指定的全部参考文献。							
(2) 外文翻译：3000 字以上。							

(3) 文献综述：3000 字以上，包括国内外研究现状、研究方向、进展情况、存在问题和参考依据等。

(4) 开题报告：3000 字以上，包括选题的意义、可行性分析、研究内容、研究方法、拟解决的关键问题、预期结果和研究进度计划等。

2、论文：10000 字以上，包括绪论、正文、结论、参考文献等。法学、人文、经管类专业 8000 字以上，特殊专业应满足分院根据培养目标另行制定的相应要求。

3、课题要解决的主要问题和具体要求（填写如下）：

一、课题定位与目标

本课题选择独立开发者维护软件项目作为具体业务场景，设计并实现一个人工智能生成内容（Artificial Intelligence Generated Content，AIGC）协作开发 workflow 平台。平台以项目目标、需求资料、代码仓库、决策记录和验收标准为事实输入，通过模型上下文协议（Model Context Protocol，MCP）连接代码智能体、Git/GitHub 及测试工具，把需求澄清、任务规划、代码执行、人工审查、测试和交付组织为可编辑、可执行、可追溯的工程闭环。

毕业设计以可运行平台原型和工程评价结果为主要成果。评价围绕四个维度展开：范围维度验证 Project Brief、需求与任务及交付物之间的覆盖关系；效率维度记录任务总耗时、迭代和人工介入；成本维度记录模型调用、Token、执行资源与部署开销；质量维度记录测试、缺陷、返工、审查和交付完整性。现有工程资料和代码只作为需求分析、方案选型与技术可行性验证的输入，不直接作为毕业设计成果；本阶段重新完成需求收敛、用户体验原型、架构设计、核心实现、测试与对照实验。

二、主要交付内容

(1) 需求规格与用户体验（User Experience，UE）核心原型。依据项目现有核心想法、开发计划和 Focus OS 界面规范，形成独立开发者的典型用户旅程、功能范围、异常场景和验收标准；完成项目创建或导入、Project Brief 确认、workflow 成果视图与完整流程、Task 工作区、Assist 协作、审批、Diff Review、资产和交付结果等核心界面原型。

(2) 工作流与项目上下文模块。采用“Project—Workflow—Workstream—Task”层级，把 Project Brief 中的功能、里程碑、风险和验收条件映射到可独立验收的任务；用 Node Contract 声明任务目标、输入、输出、允许工具和验收标准。AIGC 生成候选工作流，确定性校验器检查依赖、需求覆盖和资产流，用户确认后形成正式版本；任务执行前由系统从显式输入、直接依赖、项目决策和仓库快照构建结构化上下文包（Context Pack）。

(3) MCP 工具集成与受控执行模块。系统通过 MCP 能力目录和 Runner Adapter 接入 Codex 等代码智能体，页面、REST、MCP 和终端入口复用同一业务服务、权限与审计规则。任务在受管 Repository Workspace 及隔离 worktree 中执行，记录命令、文件变更、工具调用和状态事件；写入、敏感操作和外部交付经过人工确认、checkpoint 与 Diff Review，凭据不进入 Context Pack、Trace 或前端响应。

(4) 资产证据与工程交付模块。将需求、决策、代码变更、测试报告和交付记录保存为带版本和来源关系的 Asset，使用 Trace、Workspace Digest、内容哈希及证明记录关联任务输入、执行过程与输出。验收通过后形成 Git 提交或 Draft Pull Request，并汇总任务耗时、模型调用、人工介入、测试结果、缺陷和返工数据，作为范围、效率、成本和质量评价的证据。

三、系统架构与 AIGC 技术路线

系统前端采用 React、TypeScript 和 Vite 实现项目接入、workflow 画布、Task 工作区、Assist、审批和审查界面；后端采用 Node.js 实现项目、workflow、上下文、资产、权限、执行与交付服务，原型阶段使用 JSON-local 本地持久化并保留 Prisma 数据模型。MCP Gateway 负责 Agent 入口，Core 保留能力目录、业务状态、审批、凭据和执行权威；Runner 在 Docker 或受管本地环境中运行，通过 Context Pack 与平台解耦。

技术路线按真实开发链路推进：项目创建或导入后完成 Intake 并形成版本化 Project Brief；AIGC 依据 Brief 和仓库事实生成 Workstream/Task 候选，校验通过且经用户确认后安装正式工作流；系统为当前 Task 固定 Node Contract、输入资产、直

接依赖和仓库快照，生成 Task Execution Context 与 Context Pack；Runner 调用获准的 MCP 工具在隔离工作区执行，平台持续记录 Execution Event 与 Trace；用户在审批和 Diff Review 中处理关键决策，测试通过后绑定输出资产并形成 Git 提交或 Draft PR；最终用运行证据进行复盘和四维对照评价。

四、核心工程问题与处理重点

（1）需求与 UE 核心原型的范围一致性。Project Brief 作为需求基线，功能、验收条件、里程碑和风险均需映射到负责的 Workstream/Task；界面同时呈现目标、依据、执行状态、阻塞原因和人工决策入口，使用户能够理解并控制 AIGC 工作过程。

（2）架构设计中的上下文主权与组件可替换性。项目上下文、资产、决策和 Trace 由平台持有，代码智能体只消费按任务组装的 Context Pack；Runner Adapter 和 MCP 能力目录隔离具体模型及工具差异，REST、MCP 和网页入口统一调用领域服务。

（3）实现集成中的执行边界与状态一致性。外部代码源保持只读，写操作限定在受管 checkout/worktree；权限、路径、敏感信息、写锁、幂等、checkpoint、失败恢复和仓库快照共同约束长任务执行，避免并发覆盖、重复交付和越权访问。

（4）测试评价中的证据可比性。直接使用 AIGC 工具与平台工作流采用相近任务、相同代码基线和统一验收条件；范围覆盖、耗时、Token 与资源、测试结果、缺陷返工和审查记录均从平台运行数据或版本库证据中采集，结论同时说明样本和外部服务限制。

五、开发过程与进度要求

（1）需求与 UE 原型阶段：以核心想法、愿景范围、开发计划和现有界面规范为输入，完成场景边界、用户旅程、需求规格、非功能要求、信息架构和可交互原型，并建立需求到验收标准的追踪表。

（2）架构与最小闭环阶段：完成领域模型、模块边界、接口、权限和数据方案，优先跑通“Project Brief—Workflow—Task—Context Pack—受控执行—Diff Review—测试—交付”的最小闭环。

（3）实现集成与质量加固阶段：分阶段接入 Codex Runner、MCP、受管仓库、资产证据和 Git/GitHub 交付；每个增量配置单元、集成和端到端测试，并覆盖路径隔离、敏感信息、幂等、审批、回滚和异常恢复。

（4）实验优化与成果交付阶段：选择具有代表性的同类开发任务，对照直接使用 AIGC 工具与平台工作流；根据四维数据优化工作流、上下文和交互，完成部署包、源代码、项目文档、测试报告、实验报告和答辩材料。

六、预期成果与验收要求

（1）提交可运行的平台原型及源代码，完整演示项目接入、Brief 确认、工作流生成与校验、Task 上下文构建、受控执行、人工审查、自动化测试、资产绑定和 Git 交付。

（2）提交需求规格、UE 核心原型、系统架构与数据模型、接口说明、部署运行手册、测试用例、需求追踪矩阵和实验报告，关键结论可追溯到代码、测试、Trace、Asset 或版本库记录。

（3）系统通过静态检查、单元测试、集成测试和端到端测试，重点验证需求覆盖、上下文范围、权限路径、敏感信息、审批幂等、失败恢复、输出绑定和交付一致性；未执行的真实外部服务测试明确标记。

（4）至少完成两组规模相近、代码基线和验收条件可比的软件开发任务，对比直接使用 AIGC 工具与本平台工作流，报告范围、效率、成本和质量数据、原始证据、结果及局限。

（5）平台形成需求、决策、执行、测试和交付的证据链，正式 Task 的必需输出全部绑定并通过验收后方可完成，代码交付包含 Diff、测试结果、提交或 Draft PR 记录。

(6) 完成不少于 10000 字的毕业论文（设计说明书），包括绪论、相关工作、需求与 UE 设计、系统架构、实现、测试与工程评价、结论与展望，并按 GB/T 7714—2015 著录参考文献。

推荐参考文献：

[1] WANG L, MA C, FENG X, et al. A survey on large language model based autonomous agents[J]. Frontiers of Computer Science, 2024, 18(6): 186345. DOI:10.1007/s11704-024-40231-1.

[2] XI Z, CHEN W, GUO X, et al. The rise and potential of large language model based agents: A survey[J]. Science China Information Sciences, 2025, 68(2): 121101. DOI:10.1007/s11432-024-4222-0.

[3] WANG Y, ZHONG W, HUANG Y, et al. Agents in software engineering: Survey, landscape, and vision[J]. Automated Software Engineering, 2025, 32(2): 70. DOI:10.1007/s10515-025-00544-2.

[4] QIAN C, LIU W, LIU H, et al. ChatDev: Communicative agents for software development[C]//Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics. Bangkok: Association for Computational Linguistics, 2024: 15174-15186. DOI:10.18653/v1/2024.acl-long.810.

[5] YANG J, JIMENEZ C E, WETTIG A, et al. SWE-agent: Agent-computer interfaces enable automated software engineering[C]//Advances in Neural Information Processing Systems 37. [S.l.]: Neural Information Processing Systems Foundation, 2024: 50528-50652. DOI:10.52202/079017-1601.

[6] BARKE S, JAMES M B, POLIKARPOVA N. Grounded Copilot: How programmers interact with code-generating models[J]. Proceedings of the ACM on Programming Languages, 2023, 7(OOPSLA1): 85-111. DOI:10.1145/3586030.

[7] VAITHILINGAM P, ZHANG T, GLASSMAN E L. Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models[C]//CHI Conference on Human Factors in Computing Systems Extended Abstracts. New York: ACM, 2022: 1-7. DOI:10.1145/3491101.3519665.

[8] DODERLEIN J B, KOUADIO N H, ACHER M, et al. Piloting Copilot, Codex, and StarCoder2: Hot temperature, cold prompts, or black magic?[J]. Journal of Systems and Software, 2025, 230: 112562. DOI:10.1016/j.jss.2025.112562.

[9] CUI K Z, DEMIRER M, JAFFE S, et al. The productivity effects of generative AI: Evidence from a field experiment with GitHub Copilot[J/OL]. An MIT Exploration of Generative AI, 2024[2026-07-22]. DOI:10.21428/e4baedd9.3ad85f1c.

[10] AMERSHI S, WELD D, VORVOREANU M, et al. Guidelines for human-AI interaction[C]//Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems. New York: ACM, 2019: 1-13. DOI:10.1145/3290605.3300233.

[11] KLEPPMANN M, WIGGINS A, VAN HARDENBERG P, et al. Local-first software: You own your data, in spite of the cloud[C]//Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software. New York: ACM, 2019: 154-178. DOI:10.1145/3359591.3359737.

[12] VAN DER AALST W M P, TER HOFSTEDE A H M, KIEPUSZEWSKI B, et al. Workflow patterns[J]. Distributed and Parallel Databases, 2003, 14(1): 5-51. DOI:10.1023/A:1022883727209.

[13] MOREAU L, CLIFFORD B, FREIRE J, et al. The Open Provenance Model core specification (v1.1)[J]. Future Generation Computer Systems, 2011, 27(6): 743-756. DOI:10.1016/j.future.2010.07.005.

[14] SCULLEY D, HOLT G, GOLOVANOV E, et al. Hidden technical debt in machine learning systems[C]//Advances in Neural Information Processing Systems 28. Red Hook, NY: Curran Associates, 2015: 2503-2511.

[15] MODEL CONTEXT PROTOCOL. Model Context Protocol specification: 2025-06-18[EB/OL]. (2025-06-18)[2026-07-22]. <https://modelcontextprotocol.io/specification/2025-06-18>.

[16] WORLD WIDE WEB CONSORTIUM. PROV-DM: The PROV data model[S/OL]. (2013-04-30)[2026-07-22]. <https://www.w3.org/TR/2013/REC-prov-dm-20130430/>.

指导教师签名：_____

年 月 日

毕业论文（设计）工作指导小组意见：

☐ 同意下达任务书 ☐ 不同意下达任务书

负责人签名：_____

年 月 日